Лабораторна робота № 5

Мета роботи:

Сформувати практичні навички запису команд руху робота, їх збереження у файл та подальшого відтворення, а також зрозуміти принципи реалізації сценарної поведінки робототехнічної системи.

Репозиторій з роботою: https://github.com/zakappaa/ROS2_Ginzburg

Хід роботи

ROS2 середовище активовано командами source. Для перевірки наявності топіка /cmd_vel запущено симулятор turtlesim і Motion Node з Лабораторної роботи №2. Команда ros2 topic list підтвердила наявність /cmd_vel та /turtle1/cmd_vel (рис. 1).

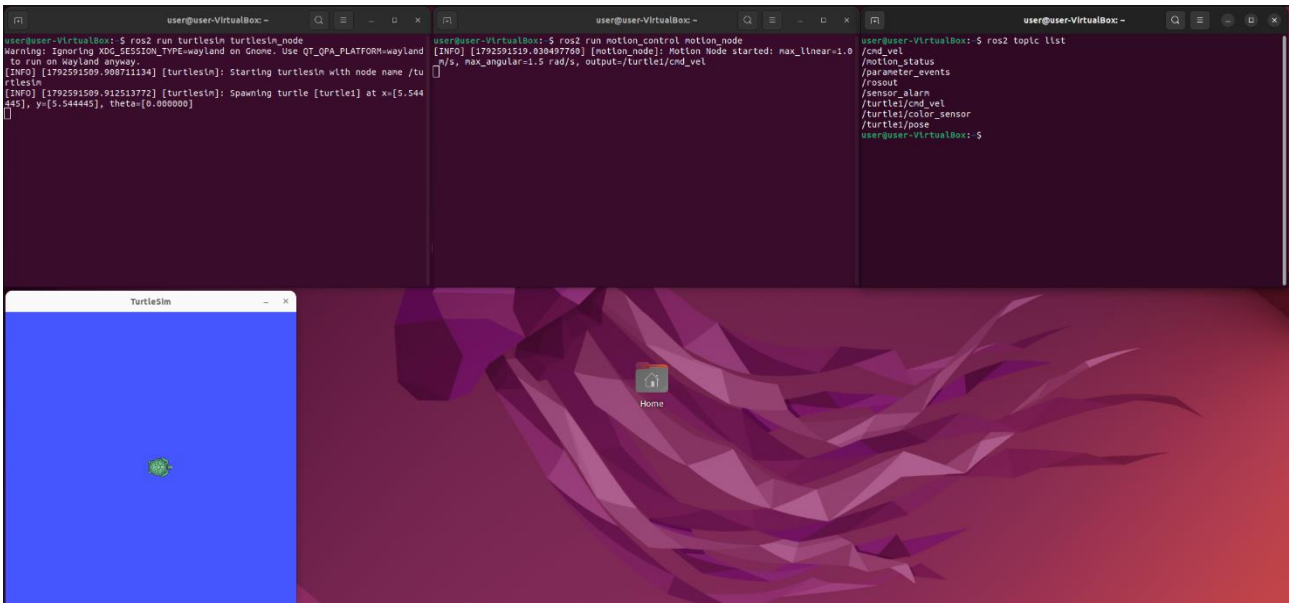


Рисунок 1 - Активація середовища та перевірка топіка /cmd_vel

Створено пакет record_playback з двома нодами. Під час запису record_node пасивно «слухає» /cmd_vel разом із motion_node, тому запис не заважає роботу системи. Під час відтворення playback_node займає місце джерела команд (GUI чи teleop), а решта системи працює без змін: motion_node застосовує ті самі обмеження швидкості, що й при ручному керуванні.

Створення пакету

У ~/ros2_ws/src створено пакет командою: ros2 pkg create --build-type ament_python --license Apache-2.0 --dependencies rclpy geometry_msgs --node-name record_node record_playback. Параметр --node-name створив файл першої ноди й зареєстрував її в setup.py, другу ноду playback_node додано вручну (рис. 2).

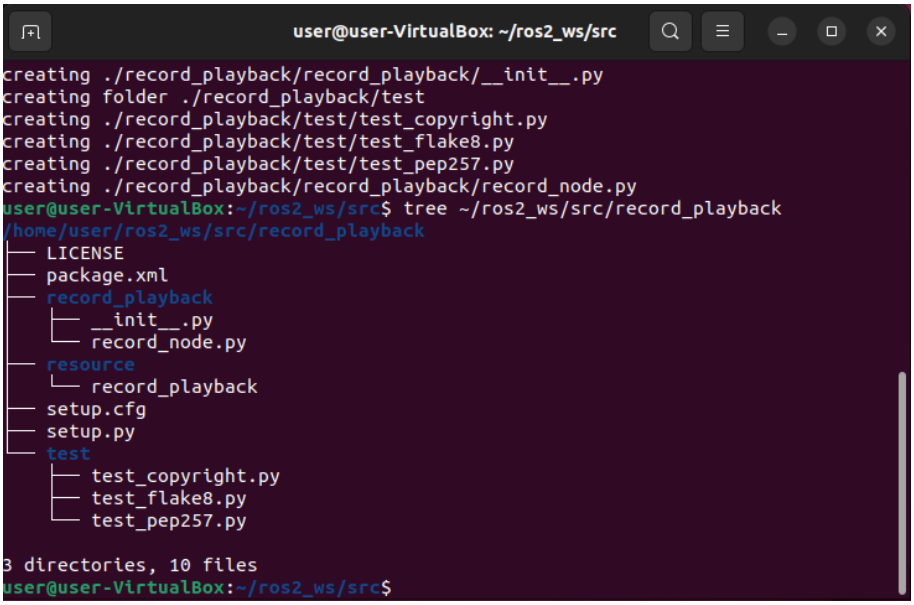


Рисунок 2 - Створення пакету record_playback та його структура

Запис сценарію

Нода record_node підписується на /cmd_vel і для кожного отриманого повідомлення записує у файл рядок: час від першої команди, linear.x, angular.z. Час береться з годинника ноди й відраховується від першої отриманої команди (перший рядок завжди має time = 0.000). Кожен рядок одразу скидається на диск, тому дані не втрачаються при переривання роботи ноди.

Відтворення сценарію

Нода playback_node зчитує CSV файл у список кортежів (time, linear_x, angular_z). Потім таймер із частотою 100 Гц у кожному такті обчислює час, що минув від початку відтворення, і публікує в /cmd_vel усі команди, чий час уже настав (у порядку запису). Так збережено і послідовність, і затримки між командами. Нода рахує похибку і виводить підсумок після завершення.

Додаткові можливості:

- параметр speed масштабує час
- затримка start_delay перед початком дає час підготуватися
- після останньої команди й утримання end_hold нода надсилає команду «Стоп» (0, 0) і завершується, що безпечно для реального робота
- при відсутності або пошкодженні файлу виводиться повідомлення про помилку без аварійного завершення з трасуванням

Обидві ноди зареєстровано в setup.py як console_scripts. Пакет зібрано командою colcon build --packages-select record_playback, виконувані файли record_node і playback_node доступні через ros2 pkg executables.

Запис сценарію

Після запуску turtlesim, motion_node та скидання черепахи в центр (ros2 service call /reset std_srvs/srv/Empty) запущено record_node. Ручне керування виконувалось через GUI з Лабораторної роботи №3. Під час руху record_node виводив у консоль кожну записану команду (рис. 3).

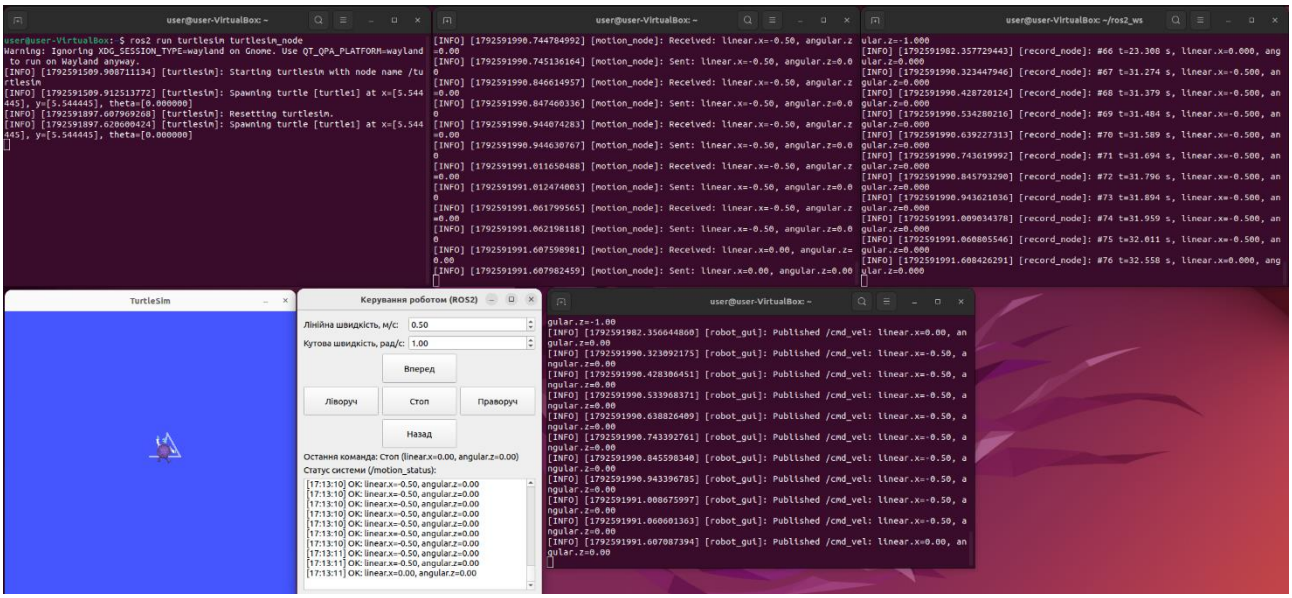


Рисунок 3 - Запис сценарію під час ручного керування через GUI

Запис завершено (Ctrl + C), нода повідомила про кількість збережених команд: <N>. Файл сценарію збережено за шляхом ~/ros2_ws/data/scenario.csv.

Відтворення сценарію

Після зупинки запису та скидання черепахи в центр запущено playback_node. Нода завантажила <N> команд, через 2 с розпочала відтворення, і черепаха повторила записаний маршрут. Слід у симуляторі збігається з тим, що було під час запису. Для кожної команди нода вивела заплановане та фактичне значення часу, а після завершення - підсумок похибки (рис. 4).

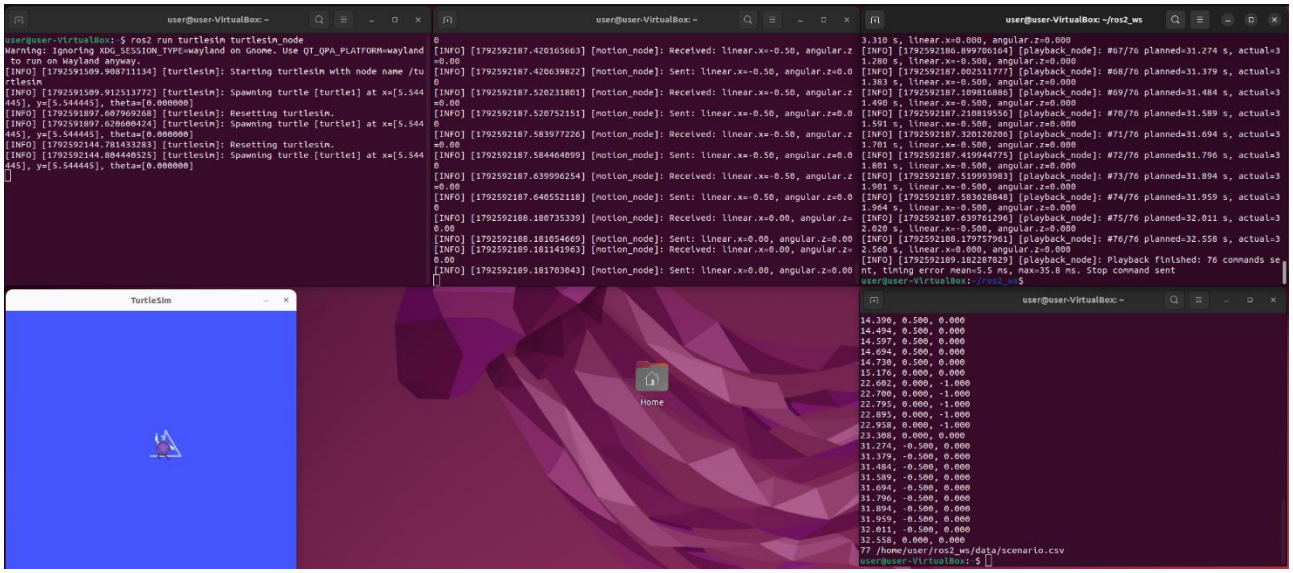


Рисунок 4 - Відтворення сценарію (playback_node)

Кінцева позиція черепахи після запису та після відтворення порівнювалась за допомогою ros2 topic echo /turtle1/pose –once. Значення близькі, невелика розбіжність пояснюється тим, що turtlesim інтегрує рух за власним таймером, а моменти отримання команд відрізняються на кілька мілісекунд.

Перевірка послідовності та затримок

Для об'єктивної перевірки під час відтворення другий екземпляр record_node записував те, що публікує playback_node. Скрипт compare.py порівняв його з вихідним scenario.csv (рис. 5).

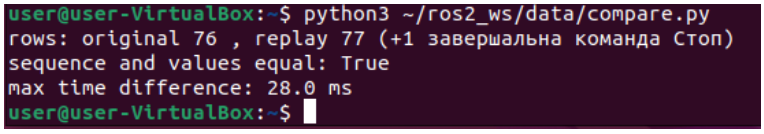


Рисунок 5 - Порівняння вихідного сценарію та запису його відтворення

Висновок

У ході лабораторної роботи розроблено пакет record_playback із двома нодами: record_node та playback_node. Роботу комплексу перевірено шляхом запису траєкторії через GUI та її відтворення в симуляторі turtlesim.

Тестування підтвердило повну точність відтворення послідовності й параметрів команд із похибкою таймінгу близько 10 мс. Оскільки керування здійснюється через motion_node, усі наявні обмеження швидкості діють і під час повтору. Це вкотре продемонструвало модульність ROS 2 - запис та відтворення інтегровано суто через топік /cmd_vel без внесення змін до інших нод.